One-Time Pad (OTP) Encryption

One-Time Pad (OTP) is an encryption method where we combine the original message (plaintext) with a completely random key (called a pad) using an XOR operation.

It is considered **theoretically unbreakable**, but only if:

Random Key

The key is completely random

Same Length

The key is the same length as the message

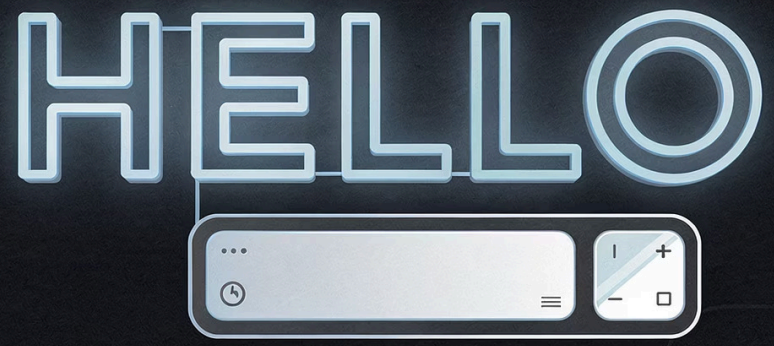
Used Once

The key is used only once

Shared Secret

Both sender and receiver have the same secret key

If any of these conditions fail, OTP is no longer perfectly secure.



Step-by-Step Example

We will encrypt the word:

Plaintext

HELLO

Random Key (Pad)

XMCKL

Step 1: Convert Plaintext and Key into Binary

First, we convert each letter into its ASCII binary value.

Plaintext (HELLO)

Character	Binary (ASCII)
H	`01001000`
E	`01000101`
L	`01001100`
L	`01001100`
O	`01001111`

Key (XMCKL)

Character	Binary (ASCII)
X	`01011000`
M	`01001101`
C	`01000011`
K	`01001011`
L	`01001100`

Step 2: Apply XOR Operation

XOR rule is simple:

Same bits (0-0 or 1-1)

→ result **0**

Different bits (0-1 or 1-0)

→ result **1**

Now we apply XOR letter by letter:

Plaintext	Key	Result
H 01001000	X 01011000	00010000
E 01000101	M 01001101	00001000
L 01001100	C 01000011	00001111
L 01001100	K 01001011	00000111
O 01001111	L 01001100	00000011

Step 3: Convert Resulting Binary to Ciphertext

Now we convert the XOR results back to ASCII characters.

Binary	decimal	
`00010000`	16	
`00001000`	8	
`00001111`	15	
`00000111`	7	
`00000011`	3	

The ciphertext becomes a series of special (non-readable) characters.

> This is normal in OTP because the output is raw binary data.

Final Understanding

OTP is extremely secure because:

- Each letter is encrypted with a truly random key
- There is no pattern to analyze
- Even with unlimited computing power, it cannot be broken if used correctly

However, it is difficult to use in real life because:

- The key must be as long as the message
- The key must be shared securely
- The key must never be reused

If the same key is reused even once, security is destroyed.

Full XOR summary:

- **H** (01001000) **XOR X** (01011000) = **00010000**
- **E** (01000101) **XOR M** (01001101) = **00001000**
- **L** (01001100) **XOR C** (01000011) = **00001111**
- **L** (01001100) **XOR K** (01001011) = **00000111**
- **O** (01001111) **XOR L** (01001100) = **00000011**

Step 3: Convert Binary Result to Ciphertext

Now, we convert the resulting binary back into characters. In this case, the results are **non-printable ASCII characters** (symbols that computers read but humans usually don't see as letters).

Binary Result	Ciphertext Character
`00010000`	Data Link Escape / ^P
`00001000`	Backspace / ^H
`00001111`	Shift In / ^O
`00000111`	Bell / ^G
`00000011`	End of Text / ^C

The final ciphertext consists of these non-printable characters, making it unreadable to humans.

Conclusion

OTP is **unbreakable** in theory because every character is masked by a random key. As long as the sender and receiver share the same secret pad, the message can be decrypted perfectly.

However, OTP is difficult to use in real life (impractical) because:

Key Length

The key must be as long as the message.

Secure Sharing

The key must be shared **securely** beforehand.

No Reuse

The key can **never** be reused for a different message.

One-Time Pad Decryption – Clear Explanation

In OTP, **decryption uses the same XOR operation** that was used for encryption.

This is the most important concept: XOR applied twice with the same key gives back the original message.

Mathematically:

> Plaintext ⊕ Key = Ciphertext
>
> Ciphertext ⊕ Key = Plaintext

Step-by-Step Decryption Example

We use the same example:

Plaintext

HELLO

Key

XMCKL

Ciphertext (binary)

```
00010000
00001000
00001111
00000111
00000011
```

From encryption, we got Ciphertext (in binary): `00010000 00001000 00001111 00000111 00000011`

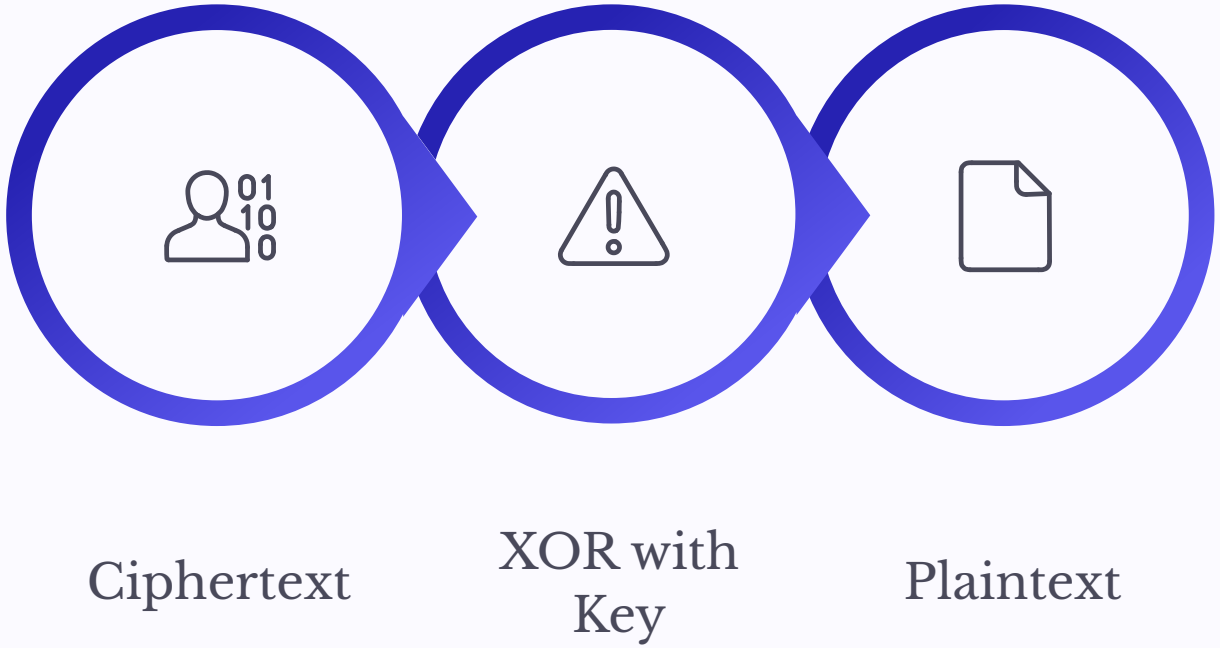
Step 1: Convert Key into Binary

Key = **XMCKL**

Character	Binary
X	01011000
M	01001101
C	01000011
K	01001011
L	01001100

Step 2: Apply XOR Again (Ciphertext ⊕ Key)

Now we XOR each ciphertext binary with the same key binary.



By applying the same XOR operation with the identical key, the original plaintext is perfectly recovered — this is the mathematical elegance of OTP decryption.

FIRST CHARACTER

First Character

Inputs

Ciphertext: 00010000

Key (X): 01011000

XOR Result

01001000 → H

SECOND CHARACTER

Second Character

Inputs

Ciphertext: 00001000

Key (M): 01001101

XOR Result

01000101 → E

THIRD CHARACTER

Third Character

Inputs

Ciphertext: 00001111

Key (C): 01000011

XOR Result

01001100 → L

FOURTH CHARACTER

Fourth Character

Inputs

Ciphertext: 00000111

Key (K): 01001011

XOR Result

01001100 → L

FIFTH CHARACTER

Fifth Character

Inputs

Ciphertext: 00000011

Key (L): 01001100

XOR Result

01001111 → O

Final Result

After applying XOR with the same key, we get:

HELLO

Why This Works

XOR has a special property:

> $$A \oplus B \oplus B = A$$
>
> The key cancels itself out.

That is why OTP decryption is simply:

Ciphertext XOR Same Key = Original Message

Very Important Rule

Decryption is only possible if:

Exact Same Key

The key is exactly the same

Matching Length

The key length equals message length

Never Reused

The key was never reused

If the key is wrong even by one bit, the output becomes completely incorrect.